Name: Siddhant Kumar **gmail:siddhant8796@gmail.com**

Assignment 23 Javascript

Q1) Explain the DOM nd its role in web development..?

Ans)The document object Model(DOM) is a programming interface for web document. It represent the web as a tree-like structure objects, where each object corresponds to an element or attribute in the web page. The DOM provides a way for JavaScript to interact with and manipulate the elements and contents of a web page.

When a web page is laoded in the browser, the browser parse the HTML and creates the DOM tree, which is a live representation of the page's structure. This tree-like structure allows javascript to interact with and modify the content,structure , and styles of the web page ,making it more interactive and dynamic

Role of DOM in JavaScipt:

1. ***Accessing HTML Elements:***
   Example:
   document.getElementById("title");
   document.querySelector(".btn");

2. ***Modifying Content Dynamically***
   Example:
   document.getElementById("msg").innerText = "Hello Siddhant!";
   document.getElementById("img").src = "photo.jpg";

3. ***Changing Styles (CSS Manipulation)***
   Example:

   document.body.style.backgroundColor = "lightblue";

4. ***Creating and Removing Elements:***
   Example:
   const p = document.createElement("p");
   p.innerText = "New paragraph";
   document.body.appendChild(p);

5. ***Handling Events***
   Exmple
   button.addEventListener("click", () => {
   alert("Button clicked!");

});

Q2)Explain the concept of event delegation and provide a scenario where it is beneficial.

Ans) **Event delegation** is a technique where you attach a **single event listener to a parent element** instead of adding separate listeners to each child element. The event is handled by leveraging **event bubbling**, where events propagate from the target element up through its ancestors.

Scenario: Click handling on a dynamic list of items

Imagine a **Todo List** where new items can be added dynamically.

Without Event Delegation (bad approach)
You would need to attach a click listener to every <li>:

- Extra memory usage
- Newly added items won't have listeners unless added again

  Example:

  HTML:

  JavaScript:

Q3)Explain the concept of Event Bubbling in the DOM.

Ans)

Event Bubbling is a mechanism in the DOM where an event starts from the target element (the element on which the event occurred) and then propagates upward through its parent elements, all the way up to the document object.

The event "bubbles up" from the child to the parent.

How Event Bubbling Works

When an event occurs (like a click):

1. The event is first handled by the **target element**
2. Then it moves to the **parent element**
3. Then to the **grandparent**

4. Continues until it reaches the **document**

HTML:

```
<div id="parent">
  <button id="child">Click Me</button>
</div>
```

Javscript:

```
document.getElementById("parent").addEventListener("click", () => {
  alert("Parent clicked");
});
document.getElementById("child").addEventListener("click", () => {
  alert("Button clicked");
});
```

Q4) Explain the purpose of the addEventListener method in JavaScript and how it facilitates event handling in the DOM.

Ans) The **addEventListener()** method is used to **attach an event handler to a DOM element**, allowing JavaScript to respond when a specific user action (event) occurs—such as a click, key press, mouse movement, or form submission.

1. Supports Multiple Event Handlers

Unlike inline onclick, you can attach **multiple listeners** to the same element.

```
button.addEventListener("click", firstFunction);
button.addEventListener("click", secondFunction);
```

2. Works with Event Propagation

It allows control over **event bubbling and capturing**.

```
parent.addEventListener("click", handler, true); // capturing
child.addEventListener("click", handler);       // bubbling
```

3. Enables Event Delegation

By attaching a listener to a parent element, it can handle events from child elements.

```
document.getElementById("list").addEventListener("click", (event) => {
    if (event.target.tagName === "LI") {
        console.log(event.target.textContent);
    }
});
```


```
document.getElementById("list").addEventListener("click", (event) => {
    if (event.target.tagName === "LI") {
        console.log(event.target.textContent);
    }
});
```


Q5)

Anwer 5) Answer

Q6)

Answer 6) Answer

Q7)

Answer 7) Answer

HTML

Javascript

Q8)

Answer 8) Answer

HTML

Javascipt